TensixFlow — A Comprehensive Report: Theory, Decisions, Policy, Orchestration, Hardware, Software, and Deployment

TensixFlow build campaign

August 3, 2026

Contents

1 Theory **4**
1.1 Tile-native spatial compute . . . 4
1.2 Operator fusion . . . 4
1.3 NoC-aware sharding . . . 5
1.4 Numerical validation theory . . . 6

2 Design Decisions **8**
2.1 Decision register . . . 8
2.2 Deviation register . . . 10

3 Verification and Honesty Policy **12**
3.1 Honesty policy: generated kernels are structurally validated only . . . 12
3.2 Layered non-vacuity defense: keeping the PCC bar honest . . . 12
3.3 Disjunctive acceptance policy . . . 13
3.4 Scope guardrails as policy . . . 15

4 Orchestration **16**
4.1 Execution model . . . 16
4.2 Wave plan . . . 17
4.3 Scale and cadence . . . 17
4.4 Final verification wave . . . 18

5 Hardware **20**
5.1 The modeled device . . . 20
5.2 The per-core L1 budget model . . . 20
5.3 Bring-up runbook (five validated steps) . . . 21
5.4 Device / no-device acceptance . . . 21

6 Software **23**
6.1 Architecture . . . 23
6.2 The `tensixflow` dialect: ops . . . 24
6.3 Passes . . . 24
6.4 Validation tiers (live counts) . . . 24
6.5 Generated-kernel validation mechanics . . . 24

7 Deployment **27**
7.1 Reproducibility: the pinned nix toolchain 27
7.2 CI topology 27
7.3 Cloud policy: opt-in, documented, never invoked 27
7.4 Upstream contribution: scouted, implemented locally, not submitted 28

8 Appendix **29**
8.1 Verification commands (F3 final wave) 29
8.2 Glossary 30
8.3 Deviation register 31

Abstract

TensixFlow is a from-scratch C++/MLIR compiler stack that lowers fused transformer operators — an RMSNorm+MatMul pair and a full MLP block — into TT-Metalium-style reader/compute/writer kernels for a modeled Tenstorrent Wormhole n150-class spatial accelerator: an 8x8 core grid, 32x32 tiles, bf16 data with fp32 accumulation, and a conservative 1 MiB modeled L1 per core. Two decisions carry the design. First, the compiler is a self-contained out-of-tree stack with its own dialect, so every stage builds and tests on an ordinary CPU machine without Tenstorrent hardware. Second, validation is deliberately split: numerical correctness is established at the schedule-IR level, where a functional tile simulator replays the lowered kernel schedule and is compared against an fp32-exact golden, while the generated kernels receive structural validation only. The pipeline is green end to end: 25 lit/FileCheck tests (24 passed, 1 gated unsupported), 11/11 CTest units, 89 pytest cases, and simulator-vs-golden Pearson correlation of at minimum 0.99998885 on the MLP block and 0.99999316 on RMSNorm+MatMul across the documented seeds, against a fixed correctness floor of 0.9875. The honesty contract is explicit: no Tenstorrent device is in the loop; the generated kernels are structurally aligned and compile-checked against signature-only stub headers pinned to tt-metal `@df36cd6`, and are never executed.

1 Theory

This section develops the four ideas the compiler is built on: tile-native spatial compute (Section 1 kernel model), operator fusion, NoC-aware sharding, and the numerical-validation theory behind the correctness bar. Every constant, byte count, and measured value quoted here is taken from the repository (`lib/tensixflow/Scheduler/L1Model.h`, `docs/performance_model.md`, `reports/rmsnorm_matmul_plan.json`, and the checked-in test evidence); nothing is quoted from outside it.

1.1 Tile-native spatial compute

The modeled device is a single Wormhole n150-class part: an 8x8 grid of 64 Tensix-style cores. Each core is a tile-native engine: it does not operate on raw tensors but on 32x32 element tiles, with bf16 data and fp32 accumulation (modeled math fidelity HiFi4). Every core owns a private slice of L1 SRAM; tt-metal-style memory addressing is `(x, y, local L1 address)`, so any core can name any other core's L1. Cores are connected by a Network-on-Chip (NoC) supporting asynchronous reads, writes, and multicast; DRAM sits off-chip behind the NoC.

A program placed on a core is decomposed into three kernel roles, matching the TT-Metalium convention:

- **Reader** (data movement): pulls input tiles from DRAM (or from a peer core's L1) over the NoC and pushes them into input circular buffers.
- **Compute**: pops tiles from the input buffers, runs the tile math (rmsnorm, matmul, gelu), and pushes result tiles into the output buffer.
- **Writer** (data movement): drains the output circular buffer and writes the tiles back over the NoC to DRAM.

The three roles never touch each other directly. They communicate exclusively through *circular buffers*: on-chip producer/consumer queues of tiles with a fixed depth. The push side reserves space (`cb_reserve_back`/`cb_push_back`) and blocks when the queue is full; the pop side waits for data (`cb_wait_front`/`cb_pop_front`) and blocks when the queue is empty. Depth is the pipelining knob: it controls how far ahead the reader may run of the compute (and the compute of the writer) before stalling.

Figure 1 shows the memory view of one modeled core. The per-core L1 budget is the single shared constant set of `lib/tensixflow/Scheduler/L1Model.h`: 1 MiB total (`kL1TotalBytes` = 1048576) minus a 64 KiB reserve for code and stack (`kL1ReserveBytes` = 65536) leaves a 960 KiB circular-buffer + shard budget (`kCBShardBudget` = 983040). One 32x32 bf16 tile costs $32 \times 32 \times 2 = 2048$ bytes (`kTileBytes`). The default three-buffer pipeline (c_0 = in0, c_1 = in1, c_{16} = out, the tt-metal index convention) then runs at depth $\lfloor 983040/(3 \times 2048) \rfloor = 160$ tiles per buffer, consuming $3 \times 160 \times 2048 = 983040$ bytes — exactly the budget, i.e. occupancy 1.0.

1.2 Operator fusion

Without fusion, every operator boundary round-trips DRAM: RMSNorm reads the activation from DRAM and writes the normalized intermediate back out, and the following MatMul re-reads that same intermediate from DRAM (plus the weight) before writing its output. Fusing the pair into one kernel keeps the intermediate resident in L1 between the two stages — the rmsnorm output never leaves the core that produced it — and erases both the intermediate DRAM write and the intermediate DRAM re-read.

For the running example `rmsnorm_matmul` (activation $[1, 2048, 4096]$ bf16, weight $[4096, 12288]$ bf16) the intermediate has the same shape as the activation, so it costs $1 \times 2048 \times 4096 \times 2 = 16\,777\,216$ bytes. That is exactly the figure the planner records in the plan JSON as `estimated_dram_reads_saved` (and `estimated_intermediate_writes_saved`): **16 777 216 bytes**

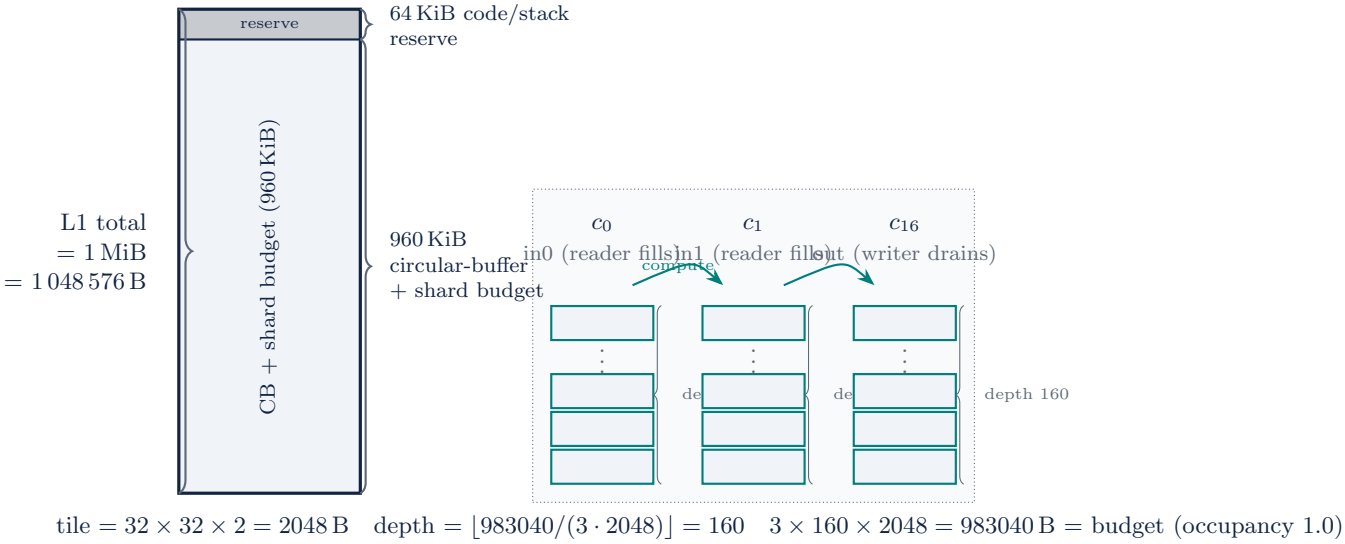


Figure 1: Memory view of one modeled core. Left: the per-core L1 budget of `L1Model.h` — 1 MiB total minus a 64 KiB code/stack reserve leaves the 960 KiB circular-buffer + shard budget. Right: the default three-buffer pipeline (c_0 in0, c_1 in1, c_{16} out) drawn as on-chip producer/consumer tile queues at depth 160; the three buffers consume exactly the budget (983040 B, occupancy 1.0). Tile size 2048 B is bf16 32x32.

each, with the eliminated tensor counted on the fused op via the `tensixflow.eliminated_bytes` / `tensixflow.eliminated_tensors` annotations. Figure 2 contrasts the two dataflows.

The fusion pass goes one step further on the full MLP chain (`examples/mlp_block.mlir`):

$$\text{rmsnorm} \to \text{matmul}(W_1) \to \text{add}(b_1) \to \text{gelu} \to \text{matmul}(W_2) \to \text{add}(\text{residual})$$

is collapsed, link by link with a single-use guard at every intermediate, into a single `tensixflow.fused_mlp` kernel — one kernel launch instead of six operators, with all intermediates circulating in L1/registers rather than DRAM.[1]

1.3 NoC-aware sharding

`--tensixflow-plan-noc-sharding` maps the tensor shapes onto the 8x8 shard grid and picks, tensor by tensor, how the data moves:

- **Activation: row-major sharding.** The leading (batch/M) dimension is row-major sharded across the grid's Y-cores, so each core row owns a contiguous band of rows and streams them tile by tile. The plan records this as `activation_sharding: "row-major"`.
- **Weight: replicate or column-shard + multicast.** The weight planner compares the weight's footprint against the per-core circular-buffer budget of Figure 1. A weight that fits is *replicated*: every core holds a full copy and multicast traffic is zero. A weight that does not fit is *column-sharded* across the grid's X-cores and *multicast* across the grid's rows via row master nodes, so it is moved once over the NoC rather than re-read from DRAM per core. The running example's weight ($[4096, 12288]$ bf16 $= 100\,663\,296$ B $\gg 960$ KiB) takes the multicast path; the plan records `weight_strategy: "multicast"`.

On top of the placement, the NoC traffic estimator produces a first-order static account of both the unfused and the fused execution. For the running example it reports unfused DRAM reads of $16\,777\,216 + 16\,777\,216 + 100\,663\,296 = 134\,217\,728$ bytes (activation + intermediate +

[1] Grounding reference (inline per the citation-free contract): arXiv:2606.09879, *Operator Fusion for LLM Inference on the Tensix Architecture* — fuses RMSNorm with matmul in on-chip SRAM to cut DRAM read/write of intermediates, and distributes inputs/weights across the mesh via NoC row/column master-node multicast. The project's fixed 0.9875 correctness floor comes from the same paper's bf16-vs-fp32 context.

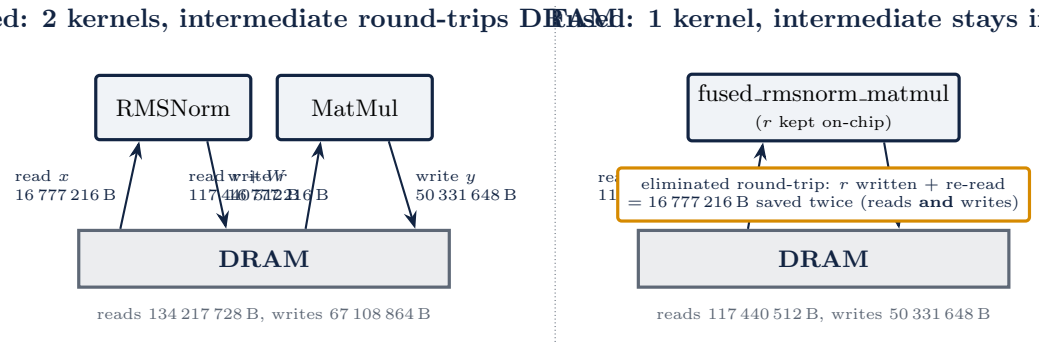


Figure 2: Unfused vs. fused dataflow for the running example. Unfused, the normalized intermediate r (16 777 216 B, same shape as the activation) is written to DRAM and re-read, adding one DRAM write and one DRAM read to the two-kernel execution. Fused, the intermediate stays in L1 between the stages: DRAM reads drop 134 217 728 → 117 440 512 B and writes 67 108 864 → 50 331 648 B — the `estimated_dram_reads_saved` = 16 777 216 figure recorded in `reports/rmsnorm_matmul_plan.json`. Byte figures are static analytic estimates from `docs/performance_model.md`, not hardware measurements.

weight) and unfused writes of 16 777 216 + 50 331 648 = 67 108 864 bytes (intermediate + matmul output), versus fused DRAM reads 117 440 512 bytes (activation + weight only, with the weight's 100 663 296 bytes counted once as NoC multicast) and writes 50 331 648, with the 16 777 216-byte intermediate held L1-resident. These estimates are also what the static performance model turns into the memory-vs-compute bound classification; every figure is a shape-derived analytic estimate, never a measured hardware number.

1.4 Numerical validation theory

The primary correctness metric is the Pearson correlation coefficient between the simulator output vector x and the golden output vector y:

$$\mathrm{PCC}(x, y) \;=\; \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\,\sqrt{\sum_i (y_i - \bar{y})^2}}, \tag{1}$$

with a **fixed correctness floor of 0.9875**: any run of either example graph below the floor is a failure, and the floor is never lowered.[2]

The comparison is deliberately *unequal-precision*: the simulator models the bf16 spatial machine while the golden is fp32-exact, so Equation (1) measures a real quantity rather than an identity. Two effects of compiled execution show up in the gap, and only in the gap:

1. **Precision cost.** The spatial machine rounds inputs to bf16, rounds every output tile back to bf16, and — inside a fused kernel — stores the intermediates between fused stages as bf16 in circular buffers (Figure 3). Each round is a loss the fp32-exact golden never pays.
2. **Tiled-execution cost.** Tile-by-tile accumulation adds partial sums in a different order than the composed whole-tensor evaluation of the golden, changing the rounding history even at fixed precision.

If the golden also rounded its intermediates to bf16, the two paths would coincide and PCC would collapse to 1.0, making any floor pass for nothing. The project therefore keeps the golden fp32-exact and guards the bar with a layered non-vacuity defense: a structural assertion that the golden's fused intermediates are fp32 while the simulator output is bf16-representable, a bit-difference check ($\max_i |x_i - y_i| > 0$, i.e. PCC is strictly below 1.0), and an empirically calibrated upper trip-wire above the measured good-case PCC. On the documented seeds the measured

[2] The floor and its context come from arXiv:2606.09879 (hardware bf16 execution versus an fp32 reference, PCC kept above 98.75%), the same grounding reference cited in Section 1's fusion subsection.

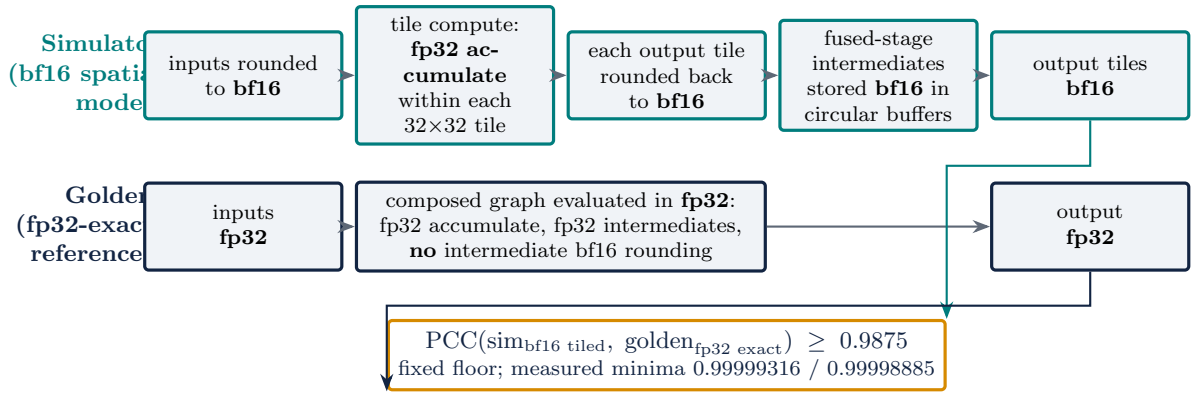


Figure 3: Precision flow under the rounding contract. The simulator lane pays the bf16 spatial cost — input rounding, fp32 accumulation within a tile, per-tile bf16 rounding, and bf16 circular-buffer intermediates between fused stages — while the golden runs the composed graph fp32-exact throughout. The PCC between the two outputs therefore measures the real numerical cost of fusion plus tiled execution, rather than collapsing to a vacuous 1.0.

minima are PCC = 0.99999316 for `rmsnorm_matmul` and PCC = 0.99998885 for `mlp_block` — far above the 0.9875 floor, and strictly below 1.0, which is exactly the band the rounding contract predicts.

2 Design Decisions

TensixFlow was planned before any code was written, and the plan kept an explicit decision register (drafts D1–D32 in the plan workspace) that was revised by three rounds of adversarial review. Table 1 records the decisions that shaped the delivered system; Section 2.2 records the points where execution deliberately and documentedly deviated from the plan text. The numerical and architectural background referenced in the register is developed in Section 1.

2.1 Decision register

Table 1: Decision register. IDs are the plan's draft-decision identifiers. "Adopted" means the decision was made during planning and the delivered system implements it; revisions note the review round that forced them.

ID	Decision	Alternatives considered	Rationale	Status
D1	Self-contained, out-of-tree compiler: own `tensixflow` MLIR dialect and a full lowering pipeline living in this repository.	Building in-tree inside Tenstorrent's tt-forge / tt-mlir repositories.	The only way to make hardware-free progress: the standalone stack builds and tests on an ordinary CPU machine with the nix LLVM 20.1.8 toolchain, needs no Tenstorrent checkout, and keeps every artifact inspectable. Compatibility with the Tenstorrent stack is handled later as a TTIR emitter plus a prose mapping doc, not by living inside their tree.	Adopted
D18	Toolchain spike first: build the upstream `llvm-project` `mlir/examples/standalone` example under the new nix flake + devenv shell before writing any dialect code.	Starting dialect work directly against the new environment.	The devenv + MLIR wiring (`MLIR_DIR`/`LLVM_DIR`/tblgen paths) was explicitly UNVERIFIED at plan time. A throwaway spike proves the wiring empirically; had it failed, the environment approach would have been revisited before any dialect investment.	Adopted; spike passed
D22	Pipeline-registration probe: prove `registerAllPasses()` plus multi-pass composition with a 2-pass lit test before relying on the full seven-pass pipeline.	Building the full pipeline first and debugging registration through it.	The synthesized pipeline-registration syntax was UNVERIFIED at plan time; the 2-pass probe isolates registration risk from pass-logic risk.	Adopted; probe passed

continued on next page

Table 1 (decision register) continued

ID	Decision	Alternatives considered	Rationale	Status
D14 (execution)	Numerical validation at the schedule-IR level: the functional simulator executes the schedule IR produced by `--tensixflow-lower-to-stream` over modeled circular buffers.	Executing the generated kernels on a device (impossible: no device owned); interpreting the emitted C++ (out of scope)	No Tenstorrent device exists in this project, so the generated reader/compute/writer/host sources are validated structurally only (golden FileCheck against current-API tokens + compile-only check) and are never executed. Keeping the numeric claim one level above the codegen artifact is what keeps the honesty disclaimer true.	Adopted
D14 (rounding)	Deliberate precision gap: simulator = bf16 spatial model (inputs bf16, fp32 accumulate within a tile, per-tile bf16 rounding, bf16 circular-buffer intermediates); golden = fp32-exact reference. See Figure 3 and Eq. (1).	Original contract — golden also rounds per-op to bf16 (rejected in review round 1, P0); a hard-coded 0.9999 upper guard (rejected in review round 2, R1).	Revised per adversarial review so the PCC measures real divergence: the original contract collapsed PCC $\to$ 1.0 and made the 0.9875 bar vacuous; the hard-coded upper guard was miscalibrated and would false-fail a correct implementation (good-case PCC exceeds it for single ops). The delivered contract makes PCC($\text{simulator}_{\text{bf16 tiled}}$, $\text{golden}_{\text{fp32 exact}}$) the measured cost of fusion + tiled execution, with non-vacuity guarded by a structural dtype assertion, a bit-difference check, and an empirically calibrated trip-wire.	Adopted after two review revisions
D4	Exactly one modeled device: Wormhole n150 class, 8x8 grid, 32x32 tiles, bf16/fp32, HiFi4, with a modeled 1 MiB L1 per core.	A multi-device target matrix; using the literal silicon budget.	A single target avoids premature generalization, and the modeled 1 MiB is a documented, conservative, tunable constant: the literal n150 L1 per core is larger ($\approx$1.5 MB), so the planner stays safely inside the real budget. Changing the one constant (and re-checking planner + model) is the supported way to target another budget.	Adopted
D3	Codegen targets the *current* tt-metal API at `@df36cd6`: single `CreateKernel` with a `variant` of `DataMovementConfig`/ `ComputeConfig`, `tt::CBIndex` circular buffers, `distributed::Enqueue*` mesh API.	Emitting the legacy `EnqueueProgram` / per-role `CreateKernel` surface.	`EnqueueProgram` no longer exists in the current `host_api.hpp`; it is kept only as a mapping note in `docs/tenstorrent_mapping.md`. The golden structural tests assert current-API tokens only, and a legacy-API fixture is deliberately kept failing so the stub set cannot silently drift back to the legacy surface.	Adopted

continued on next page

Table 1 (decision register) continued

ID	Decision	Alternatives considered	Rationale	Status
D15	A single JSON schema as the plan/perf/viz contract, owned before the consumers and imported by all of them.	Separate ad-hoc schemas per consumer (scheduler, perf model, report tool, HTML visualizer).	One schema keeps the scheduler's plan, the perf model's figures, and the visualizer's sections from drifting apart. It is deliberately *separate* from the MLIR-level schedule IR (D14), which is the codegen↔simulator contract; the two contracts serve different consumers.	Adopted
D16	Signature-only tt-metal stub headers as the substrate for the compile-only kernel check.	Building real tt-metal (its runtime requires a device); skipping the compile check entirely.	With no device, the stubs — pinned to `@df36cd6` and covering both the host-API surface and the kernel-side intrinsics — are the strongest available compile-time statement about the emitted code. The check asserts the presence of expected emitted symbols so it cannot pass vacuously, and a negative legacy-API fixture proves the stub set rejects the removed surface.	Adopted
D17	Shared L1-budget model: CB+shard budget = $L1_{\text{total}}$ − reserve, CB_bytes = $n_{\text{CB}} \times$ depth $\times$ 32 $\times$ 32 $\times$ 2, defined once in `L1Model.h` and imported verbatim by the circular-buffer allocator and the perf model.	Each component carrying its own copy of the constants.	Single source of truth: the occupancy the perf model reports is exactly the budget the allocator actually planned against (Figure 1). Duplicating the constants would let the two silently disagree.	Adopted

2.2 Deviation register

Four points where execution deviated from the literal plan text. Each deviation was documented at the time it was taken; none of them loosened a verification gate.

Table 2: Deviation register. "T¡number¿" refers to the plan's todo number whose acceptance prose was affected.

ID	Plan said	Deviation taken and why	Status
T37	Acceptance prose invoked `--emit-metalium ...-o <dir>` to write the generated project.	A bare `-o <dir>` is impossible for an MLIR pass option: `MlirOptMain` owns the global `-o` output-file flag, and `PassOptions` options are parsed only out of the pass flag's value string. Implemented and documented as `--tensixflow-emit-metalium=output-dir=<dir>` (recorded in the pass's TableGen description and the lit test header); acceptance was met in the deviated form.	Documented deviation

Table 2 (deviation register) continued

ID	Plan said	Deviation taken and why	Status
T50	Optional tt-mlir build + `ttmlir-opt` study, off the critical path.	The heavy build (tt-mlir compiles a private 30–60 min LLVM toolchain) was skipped on this box; the plan-sanctioned disjunctive OR branch was taken instead: `docs/ttmlir_study.md` documents the skip reason and carries verbatim source-study snippets from tt-mlir's own test files at `@9f06802f`.	Passed via OR branch
T55	On-device branch: compile the generated kernels against real tt-metal, run them, and validate PCC $\geq$ 0.9875 against the golden.	Skipped: there is no `/dev/tenstorrent` on this machine, so the branch is gated out by design. Acceptance is covered by the disjunctive hardware harness, which passes via its explicit if/else no-device branch (bring-up dry-run + compile-only kernel check) — the on-device branch was never silently replaced, only deferred.	Skipped (gated); not failed
T60	Submit the prepared upstream contribution as a CI-green PR (or patch + issue).	Submission skipped per the explicit owner instruction "DO NOT SUBMIT ANY PRS". The contribution was implemented and verified locally (upstream pre-commit suite green, checks run against the current upstream main) and is committed to the repository's `upstream/` tree as a documentation-only artifact; no fork, push, PR, or issue was created.	Documented only, per owner instruction

Two conventions close the register. Statuses are honest about what was *not* done: a gated skip is recorded as a skip, and a fallback branch is recorded as the fallback, never as the primary outcome. And the honesty boundary from Section 1 is preserved verbatim in every row: nothing in the register upgrades "structurally validated, never executed" to "executed" or "numerically verified".

3 Verification and Honesty Policy

TensixFlow's acceptance policy is built around one principle: *two different claims must never be conflated*. The first claim — numerical correctness — is validated at the schedule-IR level by a functional simulator. The second claim — the emitted TT-Metalium kernel sources — is validated structurally only, because no Tenstorrent device exists anywhere in this campaign's loop. This section states the honesty policy verbatim, describes the layered non-vacuity defense that keeps the numeric bar from becoming meaningless, defines the disjunctive acceptance rule for the hardware and upstream waves, and lists the scope guardrails that bound the whole project.

3.1 Honesty policy: generated kernels are structurally validated only

The project's own documentation carries a disclaimer that this report quotes verbatim rather than paraphrases, from `docs/architecture.md` (§*Honesty disclaimer*):

> "Nothing in this project executes the generated C++ kernels on hardware. There is no Tenstorrent device in the loop, and no claim of on-device behavior anywhere."
>
> "In short: the schedule is numerically validated; the generated kernels are structurally aligned and compile-checked. The two are different claims, and this project keeps them separate on purpose."

Concretely, the generated kernels (`host.cpp`, `reader.cpp`, `compute.cpp`, `writer.cpp`) pass exactly two structural layers, both executed by the test suite and never by prose:

1. **Golden FileCheck patterns against the current tt-metal API** (pinned `@df36cd6`): the patterns assert current-API tokens — the single `CreateKernel` with a `variant<ComputeConfig, DataMovementConfig>`, `tt::CBIndex::c_N` circular buffers, and the `distributed::Enqueue*` mesh API.
2. **A compile-only `-fsyntax-only` check** of every generated kernel against signature-only tt-metal stub headers pinned to `@df36cd6`. The stubs carry no behavior; they exist so that an API drift becomes a compile error rather than a silent mismatch.

The check is made non-vacuous by a deliberately failing negative fixture: a kernel that calls the *removed* legacy `EnqueueProgram` API must be rejected by the stub set. The F3 hands-on QA run logged the pair explicitly: the current-API baseline compiled, the legacy fixture failed with `EnqueueProgram` undeclared (exit 1), and all four generated kernel files compiled `-fsyntax-only` (`.omo/evidence/task-F3-tensixflow.log`, step 8). Numeric correctness, in contrast, is asserted only at the schedule-IR level, as detailed in Section 3.2 and exercised end-to-end in Section 4.4.

Policy rule for numbers. Every numeric claim in this report and in the repository (PCC values, trip-wires, saved bytes, test counts) must originate from a running test or harness invocation recorded in `.omo/evidence/`, never from prose. This is the plan's own guardrail: "No hand-waved correctness: every numeric claim (PCC, bytes-saved) must come from a running test/harness, never prose."

3.2 Layered non-vacuity defense: keeping the PCC bar honest

A similarity metric is only meaningful if it can fail for the right reasons. The similarity bar here — Pearson correlation coefficient (PCC) between simulator output and the NumPy golden — could silently degenerate in two directions: a *corrupted* implementation that happens to stay above the floor, or a *vacuous* implementation where both sides compute identically so the metric reads a meaningless perfect score. The campaign defends against both with a layered design, and the layers are ordered from structural to empirical.

Table 3: Recorded PCC band per target (16 runs total: 2 targets × 2 shapes × 4 seeds; source `.omo/evidence/task-41-tensixflow.json`). The trip-wire is the midpoint between the good-case PCC and 1.0, so it is recalibrated whenever the implementation's good case moves.

Target	Measured PCC range	Good-case PCC	Trip-wire	Floor
`rmsnorm_matmul`	0.999993159–0.999994084	0.999994084	0.999997042	0.9875
`mlp_block`	0.999988845–0.999990693	0.999990693	0.999995346	0.9875

Rounding contract (why divergence is real). The two compared paths are *deliberately* at different precision. The functional simulator is the bf16 spatial model: inputs rounded to bf16, fp32 accumulation within a tile, each output tile rounded back to bf16, and fused intermediates stored bf16 between stages. The golden reference is fp32-exact throughout the composed graph, with no intermediate bf16 rounding. PCC between the two therefore measures the real bf16-spatial-versus-exact cost of fusion plus tiled execution — and it structurally *cannot* be identically 1.0.

Primary layer — structural assertion + bit difference. Every harness run records three structural facts: the golden holds fused intermediates in fp32, the simulator output is bf16, and the golden output dtype is fp32 (fields `structural.*` in `.omo/evidence/task-41-tensixflow.json`, true on all 16 runs). On top of that, the harness asserts the bit-difference check $\max|\text{sim} - \text{golden}| > 0$: if the two outputs were ever bit-identical (the both-bf16-identical vacuous case), the run would fail deterministically. In the recorded runs the gap is far from degenerate: $\max|\text{sim} - \text{golden}|$ spans 2.77×10^{-2} to 4.13×10^{-2} across all 16 runs of both targets (`.omo/evidence/task-F3-tensixflow.log`, step 7).

Secondary layer — empirically calibrated upper trip-wire. The upper band is *not* a hard-coded constant. The trip-wire is calibrated from the running implementation on the four documented seeds (`0x20260727`, `0x5eed2026`, `0xc0ffee`, `0x1352777`): take the measured good-case PCC and set the trip-wire strictly above it but strictly below 1.0 — in practice the midpoint between the good-case PCC and 1.0. Hard-coding a value such as 0.9999 was explicitly rejected: a clean software bf16 model can exceed it, and the gate would false-fail correct code. As recorded in the metrics JSON: for `rmsnorm_matmul` the good-case PCC is 0.9999940843 and the trip-wire 0.9999970422; for `mlp_block`, 0.9999906926 and 0.9999953463. The lower bar 0.9875 is a **fixed** correctness floor: it is never lowered, and divergence is never injected to land inside the band (`docs/performance_model.md`).

Table 3 gives the full recorded picture, and Figure 4 shows the pass band geometrically, including where the two degenerate failure modes land.

The harness encodes each layer as an executable test, all green in the final hands-on QA run (11/11 in `test_correctness.py`): `test_pcc_above_floor`, `test_max_abs_diff_positive_not_bit_identical`, `test_pcc_below_empirical_tripwire`, `test_tripwire_calibrated_above_good_case`, and `test_structural_fp32_golden_bf16_simulator`, each parametrized over both targets.

3.3 Disjunctive acceptance policy

Two acceptance gates depend on resources the campaign does not control: a Tenstorrent device (not owned) and an upstream maintainer's merge decision (not owned). Both are therefore phrased *disjunctively* — each passes on a documented fallback as well as on the full path — with one strict structural requirement on how the disjunction is written.

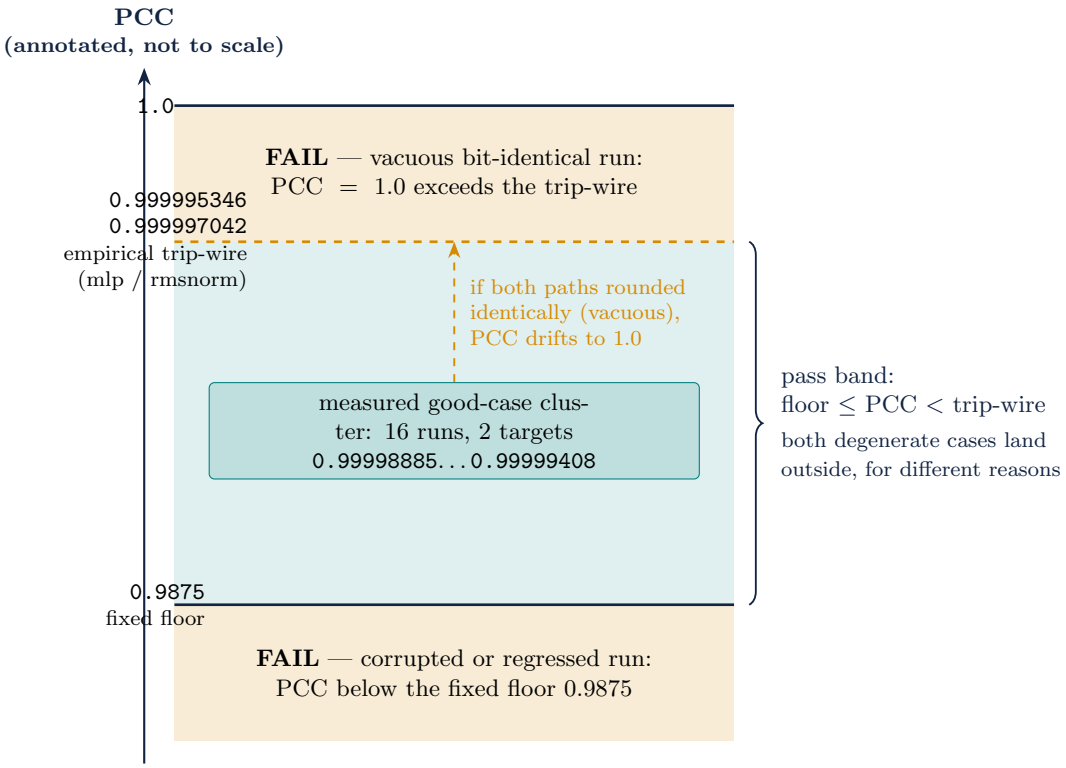


Figure 4: The non-vacuity pass band on the PCC metric. The corrupted-run failure mode (below the fixed floor 0.9875) and the vacuous-run failure mode (PCC = 1.0, bit-identical outputs) sit on opposite sides of the pass band and are caught by different layers: the floor and the empirically calibrated trip-wire respectively, with the structural fp32-golden/bf16-simulator assertion and the max $|\text{sim} - \text{golden}| > 0$ check underneath both. Axis positions are schematic; all annotated values are exact, from `.omo/evidence/task-41-tensixflow.json`.

Hardware wave: explicit `if/else`, never `&&...||`. The acceptance harness `scripts/hw_acceptance.sh` probes the device node and runs exactly one branch:

```
if [ -e "${TT_DEVICE_PATH}" ]; then
  # ON-DEVICE branch (todo 55): compile + run generated kernels,
  # validate PCC >= 0.9875 vs the NumPy golden.
  bash scripts/hw_run_on_device.sh
else
  # NO-DEVICE branch (todo 56): runbook dry-run + compile-only check.
  bash scripts/hw_bringup_dryrun.sh \
    && cmake --build build --target check-generated-kernels-compile
fi
```

Listing 1: Disjunctive hardware acceptance shape — verbatim skeleton from `scripts/hw_acceptance.sh` (`TT_DEVICE_PATH` defaults to `/dev/tenstorrent`).

The `if/else` is deliberate. An `on-device &&... || no-device` form would let a *present-but-failing* device run fall through to the passing no-device branch and be silently masked; with `if/else` exactly one branch runs and the script's exit status is that branch's status, unmasked. The on-device arm fails honestly even in absence of its entry point: if `hw_run_on_device.sh` is missing or non-executable the script exits 1 and refuses to fall through. On this campaign's CPU-only box the no-device arm is the active one and did real work: bring-up dry-run exit 0 plus `check-generated-kernels-compile` exit 0 (baseline compiles, legacy fixture rejected, four generated kernels compile) — see the final-wave QA in Section 4.4. The on-device branch itself (todo 55) was skipped and is recorded as such in the ledger, because no device is owned.

Table 4: Scope guardrails binding the campaign, and the check that enforced each.

Surface	Guardrail (binding)	Enforced / verified by
Python DSL	Thin MLIR-text builder only; *no* tracing JIT, no AST introspection	Canonicalized-IR equivalence test vs the handwritten inputs (ignoring SSA names); F4 import audit: no `ast`/`inspect`
TT-MLIR compat	TTIR *emitter* only; the other four dialects are prose mappings	`--tensixflow-emit-ttir` lit test; mapping doc covers TTIR/TTNN/D2M/TTKernel/TTMetal in prose
Remote-box script	`bash -n` syntax check + docs only; never live provisioning, no cloud spend	`bash -n scripts/remote-box.sh` exit 0; script never executed; pricing prose dated "as-of 2026-07-21"
Benchmarks	Timing loops, *not* a framework	No benchmark-framework dependency (`find_package(benchmark)` absent); each binary prints an explicit static-estimate disclaimer
HTML visualizer	Static, offline, zero external references	Playwright load with all network requests aborted: 0 requests, 0 console errors, 6/6 section anchors; 0 `http(s)` occurrences in the source
Owner constraints	nix/devenv shell binding the whole build; no NVIDIA GPU; no PR submission (T60 documented-only)	Every gate runs under `nix develop`; F3 probes: no `/dev/tenstorrent`, no NVIDIA in `lspci`; F4 audit: nothing submitted/forked/merged

Upstream wave: submitted, never "merged". The upstream contribution gate accepts either an open pull request with the *contributor's own* checks green, or a prepared `0001-*.patch` plus an issue URL. It never depends on the maintainers' CI and never claims a merge, since neither is owned by this campaign. In the event, the campaign owner issued an explicit do-not-subscribe/do-not-submit override, so todo T60 landed as *documented-only*: the scouted target, the contribution, and the submission procedure are committed under `upstream/`, and the ledger records "T60 skipped — submission process documented only."

3.4 Scope guardrails as policy

The plan's "Must NOT have" list is enforced as acceptance policy, not as advice: each stretch surface has a binding guardrail and an executable or auditable check. Table 4 reproduces them together with how each was verified during the campaign (final-wave evidence cited where applicable; see Section 4.4).

Two further guardrails shape everything downstream: exactly *one* modeled device (Wormhole n150-class: 8×8 grid, 32×32 tiles, bf16/fp32, 1 MiB modeled L1 per core) — no multi-target support — and no NVIDIA/CUDA path at all, since this is a spatial-accelerator compiler and the campaign owns no NVIDIA GPU. Both were re-audited adversarially by the F4 scope-fidelity review (repo-wide greps for `cuda|nvidia`, `grayskull|blackhole|n300`, and multi-device tokens came back clean except a single sanctioned absence-statement in a test docstring). Hardware specifics are covered in Section 5.

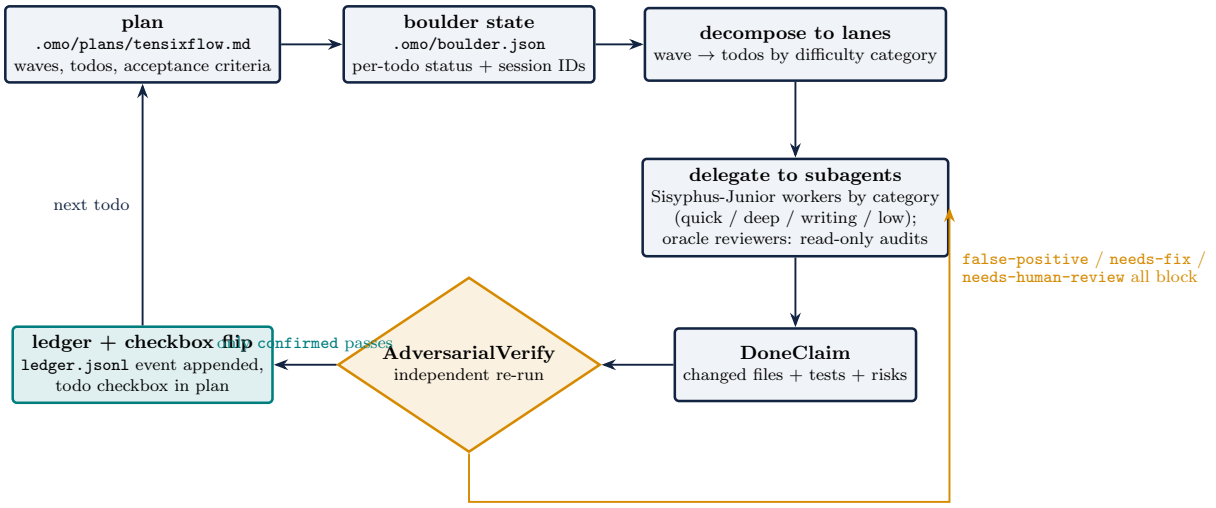


Figure 5: The campaign execution loop. Each todo moves plan → boulder state → lane decomposition → subagent delegation, returns a `DoneClaim`, and is accepted only when an independent `AdversarialVerify` run re-confirms it; only the verdict `confirmed` appends a `ledger.jsonl` event and flips the plan's checkbox. The three blocking verdicts route the todo back to delegation for rework. Structure from `.omo/start-work/ledger.jsonl` and `.omo/boulder.json`.

4 Orchestration

This section describes how the campaign was actually run, which is as much a deliverable as the compiler itself: a single orchestrator session drove every todo through a fixed execution loop, eleven parallel waves with two sanctioned skips, and a four-reviewer final verification wave. Every timing and count quoted here traces to `.omo/boulder.json`, `.omo/start-work/ledger.jsonl`, or `.omo/evidence/`; none is reconstructed from memory. The policy that bounded the work (honesty, non-vacuity, disjunctive acceptance, scope guardrails) is in Section 3.

4.1 Execution model

The campaign ran as a closed loop over the plan's 60 todos (Figure 5). The loop's state is not prose: the plan file `.omo/plans/tensixflow.md` is the single source of truth for waves, todos, and agent-executable acceptance criteria, and `.omo/boulder.json` mirrors it as machine state (active work ID, per-todo status, session IDs, and per-task `elapsed_ms` timings). Each todo was decomposed into a lane assignment, delegated to a subagent, and accepted back only through an independent verification gate.

Two properties make the loop trustworthy rather than ceremonial. First, every verification is a re-run, not a re-read: the orchestrator independently executed each todo's acceptance commands (for example, `check-tensixflow`, `ctest`, `pytest`, and the full lowering pipeline) rather than trusting the worker's summary. Second, every task-completed ledger event carries an *adversarial-class* map: the same nine trigger-mapped QA classes, assessed per task. The classes are malformed input, prompt injection, cancel/resume, stale state, dirty worktree, hung/long commands, flaky tests, misleading success output, and repeated interruptions; each event records either a concrete probe ("probed: …") or an explicit "not-applicable" with the reason. The probes are real work, not labels: the toolchain spike probed stale nix state and a dirty worktree, the first lit test carried a deliberate wrong-CHECK negative proof, the CI workflow was grepped for forbidden GPU/device tokens, and every early task verified that malformed input produced a clean diagnostic rather than a crash (see `.omo/evidence/task-1-tensixflow.log` through `.omo/evidence/task-7-tensixflow.txt`).

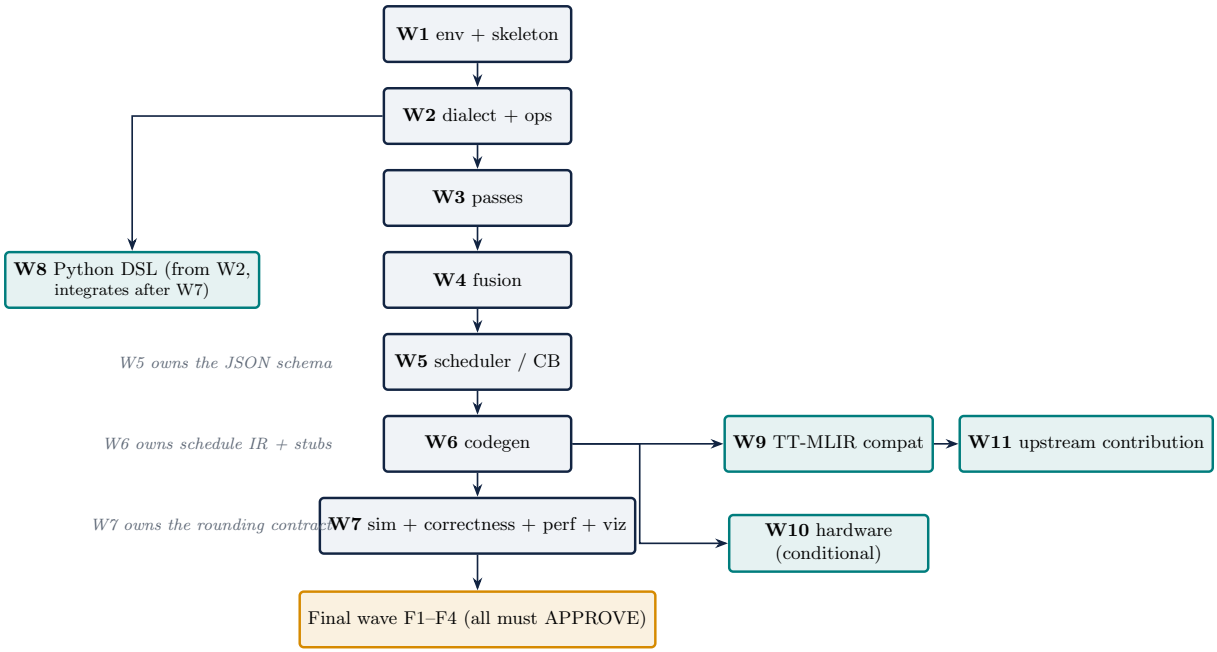


Figure 6: The eleven-wave dependency DAG plus the final verification wave. The W1–W7 spine is the critical path; W8 starts from W2 and integrates after W7, W9 and W10 wait on W6, and W11 waits on W9. Three cross-cutting contracts are owned one wave each: W5 owns the plan/perf/viz JSON schema, W6 owns the schedule IR and the signature-only stub headers, and W7 owns the bf16-simulator / fp32-golden rounding contract (Section 3.2). Dependency matrix from `.omo/plans/tensixflow.md`.

4.2 Wave plan

The 60 todos were executed as eleven parallel waves (Figure 6). The critical spine is W1 (environment and skeleton, the gating uncertainty) through W7 (simulator, correctness, perf model, visualizer, benchmarks, and docs). Four side lanes hang off the spine: W8 (the Python DSL) may begin once W2's dialect lands and integrates after W7; W9 (TT-MLIR compatibility) and W10 (the conditional hardware path) both wait on W6's codegen; W11 (the upstream contribution) waits on W9's familiarity. The Final Wave F1–F4 runs after all todos and gates the whole campaign.

The wave plan carried three cross-cutting contracts that were decided once and imported by every downstream consumer, never duplicated (Section 3): W5 owns the plan/perf/viz JSON schema, W6 owns the MLIR schedule IR (the codegen↔simulator contract) together with the signature-only tt-metal stub headers pinned at `@df36cd6`, and W7 owns the rounding contract that makes the PCC bar non-vacuous. Two todos were plan-sanctioned skips rather than failures: T55 (on-device branch) because no device is owned, covered by the disjunctive hardware acceptance (Section 3.3), and T60 (submit) because the owner issued an explicit do-not-submit override; both are recorded as skips in the ledger and produced no commits.

4.3 Scale and cadence

Per-task elapsed times were tracked in `.omo/boulder.json` as `elapsed_ms` fields. Table 5 lists the notable ones. The honest caveat on the headline number bears repeating in prose: the 176-hour campaign figure is wall-clock time from the first plan-approved state to the final wave, so it includes idle time and human-wait time (the upstream scout at 11h16m, for example, was the elapsed span of a long-running session, not 11 hours of continuous compute). The per-task figures are the reliable signal: even the heaviest implementation todos (T1, T12, T30, T39, T59) measured in minutes to a few hours of wall-clock each.

Table 5: Notable per-task elapsed wall-clock times from `.omo/boulder.json` (`elapsed_ms` fields, converted). `T<number>` refers to the plan todo.

Todo	Task	Elapsed	Note
T1	Toolchain spike: nix flake + devenv on `llvmPackages_20`	23 m 54 s	Gating uncertainty; proved the env wiring first
T12	Consolidated shape-inference + verifier test suite	13 m 6 s	Wave 2
T30	`plan-noc-sharding` + `allocate-circular-buffers` passes + plan report	10 m 44 s	Wave 5; first consumer of the JSON schema
T39	Functional tile simulator over circular buffers	15 m 29 s	Wave 7; executes the schedule IR
T58	Scout a concrete small upstream contribution	11 h 16 m	Longest single task (repo reconnaissance)
T59	Implement the contribution per upstream style + tests	3 h 17 m	Wave 11; `upstream/` artifact
Campaign	Full plan-to-final-wave wall-clock	176 h 22 m 13 s	*includes idle / human-wait time, not pure compute*

Three repository counts, all measured by direct command on this box, anchor the "one commit per todo, one evidence file per task" claim:

- `git rev-list --count HEAD` returns **58** commits: one conventional commit per implemented todo, each leaving the tree green. The 60-todo plan minus the two sanctioned skips (T55, T60) is exactly 58, and `git log --oneline` spot-checks the style, e.g. `feat(upstream): implement contribution per upstream style + tests`, `test(hw): disjunctive hardware acceptance harness`, `feat(sim): functional tile simulator over circular buffers`.
- `ls .omo/evidence | grep -c task-` returns **64** evidence files: every implemented todo 1–59 (excluding the two skips) has one, T41 and T43 additionally keep a metrics JSON and a rendered HTML, the F3 QA log and its two screenshots are archived, and the report pipeline's own evidence files live there too.
- `wc -l < .omo/start-work/ledger.jsonl` returns **68** lines: the work-started event, one task-completed or task-skipped event per todo, four wave-complete summaries, and four final-wave-verdict events (Section 4.4).

4.4 Final verification wave

After all todos landed, four independent read-only oracle reviewers ran in parallel and each returned APPROVE (Table 6). Each reviewer's session ID is recorded in `.omo/boulder.json` under `works.tensixflow-f194c7b3.session_ids`, and each verdict is logged in `.omo/start-work/ledger.jsonl` as a `final-wave-verdict` event. The wave was read-only by guarantee: `git status` after the wave showed only untracked `.omo/` evidence, and zero product files were modified by reviewers.

The campaign's direct session (`ses_0780d1f23ffeVINbwloM0T7ZwW`) is the orchestrator session shared across the boulder works; the four F-wave reviewers are the appended sessions listed above. The F3 verdict is also preserved as evidence: `.omo/evidence/task-F3-tensixflow.log` (with the two `task-F3-screenshot-*.png` renders) records every step's exit code, the captured PCC values for both targets, and the byte-identical report regeneration. The F1–F4 approvals are the acceptance events the plan's final verification wave requires.

Table 6: Final verification wave: four independent read-only oracle reviewers, all APPROVE. Session IDs are from `.omo/boulder.json works.tensixflow-f194c7b3.session_ids`; the four `final-wave-verdict` events are in `.omo/start-work/ledger.jsonl`.

ID	Reviewer role	Session ID	Read-only verification	Verdict
F1	Plan compliance audit	`ses_0361ca41cffeQDZJY0cmSiTjsD`	Re-runs every todo's acceptance command: `check-tensixflow` (25 = 24 passed + 1 unsupported), `ctest` (11/11), `pytest` (89 passed), and the full lowering pipeline on both examples (rc=0, 6 files each)	**APPROVE**
F2	Code quality review	`ses_0361c328dffeo2zvLAMhPVZbkc`	Malformed/truncated IR and malformed JSON produce clean diagnostics, never crashes; codegen emits the current tt-metal API only (grep for `EnqueueProgram` emission sites is zero); stubs signature-only pinned `@df36cd6`; ruff advisory pass	**APPROVE**
F3	Hands-on QA	`ses_0361bc016ffeEqisSH5eTHC1S0`	Build → lit → ctest → pytest → pipeline on both examples → report regeneration (byte-identical) → simulator-vs-golden harness (PCC ≥ 0.9875 both targets, non-vacuity holds) → `scripts/hw_acceptance.sh` no-device branch → network-blocked Playwright render (0 requests, 6/6 anchors)	**APPROVE**
F4	Scope fidelity	`ses_0361b41aaffe7faVAxXHZYusN4`	Adversarial greps for `cuda|nvidia`, multi-device tokens, and submitted/merged-PR claims; op-zoo and `L1Model.h` constants audits; DSL import audit (no `ast`/`inspect`/`torch`); upstream tree audit (nothing submitted, forked, or merged)	**APPROVE**

One honest sentence on this document: this report is itself a follow-up deliverable, written and compiled *after* plan completion by a worker outside the boulder loop, using an ephemeral `nix shell nixpkgs#texlive.combined.scheme-full` toolchain (`report/BUILDING.md`) that resolved against the same pinned nixpkgs rev without modifying the project flake, and its commit is therefore not counted among the 58 implementation commits of the campaign.

5 Hardware

TensixFlow targets a modeled Tenstorrent Wormhole n150-class spatial accelerator. The development box is CPU-only: *no Tenstorrent device is owned*, and nothing in the MVP touches hardware. This section documents the device model the scheduler reasons about, the always-available no-device validation, and the gated path that opens if a real n150 ever appears.

5.1 The modeled device

The device model lives in a single header, `lib/tensixflow/Scheduler/L1Model.h`, which is the declared *single source of truth* for the per-core L1 budget: the circular-buffer allocator and the static performance model import these constants verbatim and are forbidden from duplicating them. The exact constants are reproduced in Table 7.

Table 7: Modeled device specification. Constants are the exact values from `lib/tensixflow/Scheduler/L1Model.h`; the remaining rows are stated in `docs/hardware_path.md` and `scripts/hw_bringup_dryrun.sh`.

Parameter	Value	Grounding
Device class	Wormhole n150-class, single part	README, `L1Model.h` header
Core grid	8×8 Tensix grid (64 cores)	`L1Model.h` header comment
Tile shape	32×32 elements	`kTileBytes` definition
Data formats	bf16 (storage), fp32 (accumulate)	`L1Model.h` header comment
Math fidelity	`MathFidelity::HiFi4`	`docs/hardware_path.md`, Step 1
`kL1TotalBytes`	$1024 \times 1024 = 1{,}048{,}576$ B (1 MiB/core)	`L1Model.h`
`kL1ReserveBytes`	$64 \times 1024 = 65{,}536$ B (64 KiB, code/stack)	`L1Model.h`
`kCBShardBudget`	`kL1TotalBytes` − `kL1ReserveBytes` = 983,040 B (960 KiB)	`L1Model.h`
`kTileBytes`	$32 \times 32 \times 2 = 2{,}048$ B	`L1Model.h`

The modeled L1 per core is **conservative by design**: the literal Wormhole n150 L1 is larger (about 1.5 MB), but TensixFlow models 1 MiB and documents the caveat directly in the header comment. Every plan produced by the sharding and CB-allocation passes is therefore provably within budget on the real part if it fits the model.

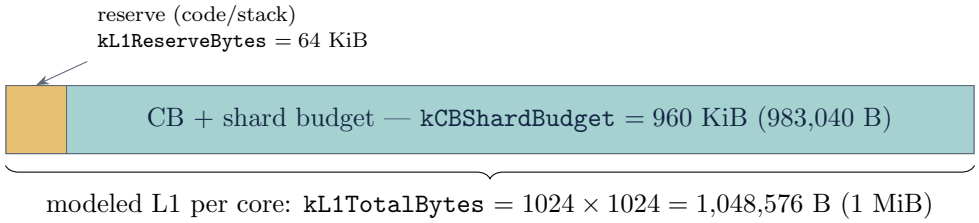
5.2 The per-core L1 budget model

Figure 7 renders the budget arithmetic exactly as the constants define it: tt-metal addresses memory by $(x, y,$ local L1 address), and each core's L1 is carved into a 64 KiB reserve (kernel code and stack) plus the 960 KiB the planner may spend on circular buffers and tensor shards. The circular-buffer sizing formula, in bytes, is

$$\text{bytes(CB)} = n_{\text{CB}} \times \text{depth} \times 2048, \tag{2}$$

i.e. the number of buffers times the depth in 32×32 tiles times `kTileBytes` = 2048 B. The committed plan `reports/rmsnorm_matmul_plan.json` instantiates this for the fused RMSNorm+MatMul

kernel: three CBs (`c_0`, `c_1`, `c_16`, the tt-metal `tt::CBIndex` convention for in0/in1/out) at depth 160 tiles each, so $3 \times 160 \times 2048 = 983{,}040$ B — exactly the 960 KiB `kCBShardBudget`, fully consumed.



CB sizing: bytes $= n_{\text{CB}} \times \text{depth} \times 2048$
rmsnorm_matmul plan: $n_{\text{CB}} = 3$ (`c_0`, `c_1`, `c_16`), depth = 160 tiles $\Rightarrow 3 \times 160 \times 2048 = 983{,}040$ B = 960 KiB

Figure 7: Per-core L1 budget of the modeled Wormhole n150-class device, per `L1Model.h`: a 64 KiB reserve carved out of the 1 MiB total leaves the 960 KiB circular-buffer/shard budget the planner spends, together with the CB sizing formula and the fused RMSNorm+MatMul working point from `reports/rmsnorm_matmul_plan.json`.

5.3 Bring-up runbook (five validated steps)

`scripts/hw_bringup_dryrun.sh` is the always-available half of the hardware acceptance: it *prints* the validated bring-up runbook (grounded in tt-metal `INSTALLING.md` at the pinned revision `@df36cd6`), probes `/dev/tenstorrent`, and always exits 0 — a dry-run, not a probe gate. Its five steps, summarized, are Table 8.

5.4 Device / no-device acceptance

The hardware wave is accepted *disjunctively*: either a run on a real device with PCC ≥ 0.9875 against the golden, or the validated no-device substitute (bring-up runbook + compile-only kernel check + hardware-gated CI). The disjunction is wired in `scripts/hw_acceptance.sh` as an **explicit if/else, never `&&...||`**:

```
if [ -e "${TT_DEVICE_PATH}" ]; then # default: /dev/tenstorrent
  # ON-DEVICE branch: compile + run + PCC >= 0.9875 vs the golden.
  # Only this arm runs on a device box; a failure is never masked.
  bash scripts/hw_run_on_device.sh
else
  # NO-DEVICE branch: runbook dry-run + compile-only kernel check.
  bash scripts/hw_bringup_dryrun.sh && \
    cmake --build build --target check-generated-kernels-compile
fi
```

The `&&...||` spelling would be unsafe: in `on-device && run || no-device`, a device that is *present but failing* would fall through to the passing no-device branch and its failure would be masked. With `if/else` exactly one branch runs and the script's exit status is that branch's status, unmasked; the harness also fail-honestly exits nonzero if the on-device arm is reached without its entry point (`scripts/hw_run_on_device.sh`) present.

The no-device branch does real work. It is not a no-op. First, the dry-run runbook exits 0. Second, `check-generated-kernels-compile` (mechanics in Section 6.5) asserts three things: the current-API baseline kernel compiles `-fsyntax-only` against the stubs, the legacy fixture referencing the removed symbol `EnqueueProgram` is *rejected* with a diagnostic naming that

Table 8: The five validated bring-up steps printed by `scripts/hw_bringup_dryrun.sh` for a Wormhole n150 box, from zero to a running generated kernel project.

Step	Summary (as the script prints it)
1. Obtain/access an n150	Shell access to a Linux box with a Wormhole n150 seated (PCIe); the modeled target: 8×8 Tensix grid, 32×32 tiles, bf16.
2. KMD + host tools	TT-Installer (`install.sh` v2.1.0, no container runtime): installs TT-KMD ($\geq$ v2.8.0), `tt-smi`, `tt-flash`. Once the KMD loads, `/dev/tenstorrent` exists — the probe target the whole acceptance keys on.
3. Build tt-metal with runtime	Clone `tt-metal`, `git checkout df36cd6`, run `./build_metal.sh`. The runtime (host API + device dispatch) is required to enqueue kernels; TensixFlow's stub headers are only for the no-device structural check.
4. Flash firmware	Flash before first use: TT-Installer drives `tt-flash`; Wormhole boards pair with `fw_pack-19.8.1.fwbundle` (firmware v19.8.1). Verify card, FW version, and health with `tt-smi -s`.
5. Run the generated project	Build `kernels/generated/fused_rmsnorm_matmul/` (host/reader/compute/writer.cpp) against the *real* tt-metal, enqueue via `distributed::EnqueueMeshWorkload`, read back, and validate PCC $\geq$ 0.9875 against the NumPy golden — the same bar as the simulator harness.

symbol, and all four generated kernel files (`host.cpp`, `reader.cpp`, `compute.cpp`, `writer.cpp`) compile. A typo in any generated kernel fails this branch honestly.

The hardware-gated CI job. `.github/workflows/ci-hardware.yml` adds a single `on-device` job, gated **twice** so it can never run on a deviceless box:

1. *Label/dispatch gate (job-level `if`):* the PR must carry the `hardware.present` label, or the run must be an explicit `workflow_dispatch` manual override.
2. *Runner gate (`runs-on`):* `[self-hosted, tenstorrent, wormhole-n150]` — only a self-hosted runner tagged as carrying the hardware can pick the job up; GitHub-hosted runners never match.

When both hold, the job first asserts `/dev/tenstorrent` exists and exits 1 on absence — a gated job that lands on a deviceless runner is a misconfiguration that must fail honestly, not silently pass — and then runs the on-device entry point, failing loudly while that script does not yet exist. In short: *a skipped-on-device is never a silent pass.* The ordinary CPU-only CI (Section 7.2) is untouched and runs on every push.

6 Software

TensixFlow is a from-scratch C++/MLIR compiler stack: a custom `tensixflow` dialect with tensor-level and fused ops, a tile-level *schedule IR*, seven lowering passes, a TT-Metalium-style kernel emitter, a functional tile simulator, a static performance model with an offline HTML visualizer, and a Python DSL plus a NumPy golden/correctness harness. Everything builds and runs on a CPU-only box.

6.1 Architecture

Figure 8 is the system map. The spine is `tensixflow-opt`: hand-written MLIR (or MLIR text emitted by the Python DSL) runs through five lowering passes, ending in the schedule IR. That IR is the pivot: the emitter turns it into generated TT-Metalium-style kernel projects (validated *structurally only*), while the functional tile simulator executes it directly (validated *numerically* against an fp32-exact NumPy golden). A side track exports the plan as JSON, runs the static performance model, and renders the self-contained HTML report; a compat track emits TTIR-shaped text.

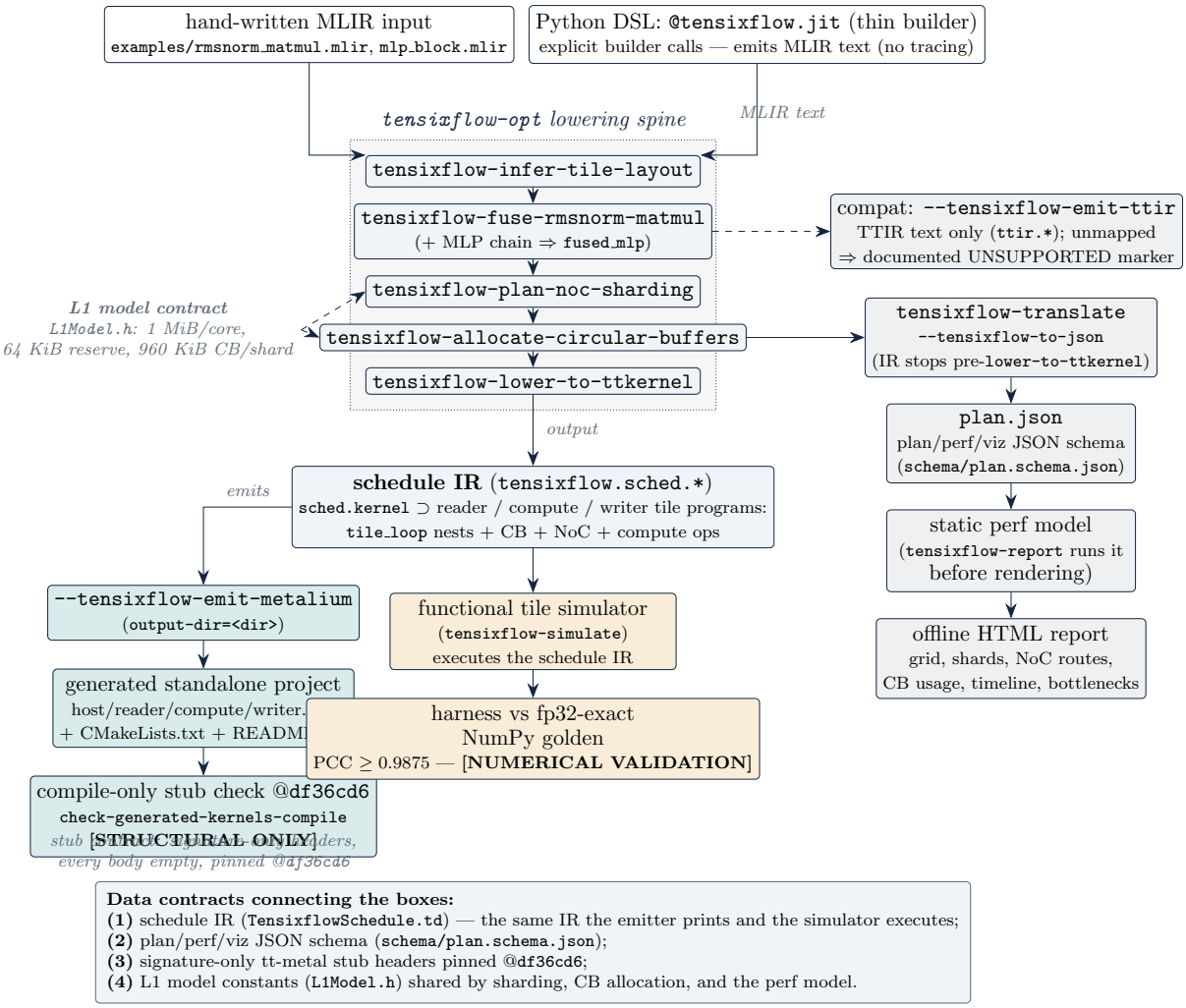


Figure 8: TensixFlow architecture: MLIR input and the Python DSL feed the `tensixflow-opt` lowering spine; the schedule IR is consumed twice — by the Metalium emitter (structural validation) and by the functional tile simulator (numerical validation). The side track exports `plan.json`, runs the perf model, and renders the offline HTML report; the compat track emits TTIR-only text.

6.2 The `tensixflow` dialect: ops

All ops live in `include/tensixflow/TensixflowOps.td` (Table 9); the schedule IR's `sched.*` ops are defined separately in `TensixflowSchedule.td` and enumerated in the Figure 8 caption above.

Table 9: Ops of the `tensixflow` dialect (tensor level), with their verifier / shape-inference status exactly as declared in `TensixflowOps.td`.

Op	Semantics	Verifier / shape inference
`func`	Function container; isolated from above, single block, implicit `return` terminator	FunctionOpInterface, Symbol; custom assembly
`return`	Terminates `func`; Pure, ReturnLike	—
`constant`	SSA value from a typed attribute	`hasFolder` (constant folding)
`add`	Elementwise sum of same-shape tensors	SameOperandsAndResultType; `hasCanonicalizer` (identity-add removal)
`gelu`	Elementwise GELU/SiLU activation	SameOperandsAndResultType
`rmsnorm`	$y = x \cdot \mathrm{rsqrt}(\mathrm{mean}(x^2) + \epsilon) \cdot \gamma$ over the last dimension	`hasVerifier`; `inferReturnTypeComponents`
`matmul`	$[..., M, K] \times [K, N] \to [..., M, N]$	`hasVerifier`; `inferReturnTypeComponents`
`fused_rmsnorm_matmul`	$\mathrm{matmul}(\mathrm{rmsnorm}(x, \gamma), w)$ as one kernel	`hasVerifier`; `inferReturnTypeComponents`
`fused_mlp`	$(\mathrm{gelu}(\mathrm{matmul}(\mathrm{rmsnorm}(x, \gamma), w_1) + b_1)\, w_2) +$ residual as one kernel	`hasVerifier`; `inferReturnTypeComponents`

6.3 Passes

Registered by `tensixflow-opt` (Table 10); the CLI chain used end-to-end is the five-pass spine of Figure 8 followed by `--tensixflow-emit-metalium=output-dir=<dir>`.

The lit suite also carries one TTIR parse test that feeds the emitted text to a real `ttmlir-opt`; since tt-mlir's toolchain is not built on this box, that test is gated and reports UNSUPPORTED (Table 11, tier 1).

6.4 Validation tiers (live counts)

All counts were re-run on this box for this report and reproduce the F3 final-wave results (Appendix, Table 12).

Two notes on tier 3. The harness runs under `uv`'s ephemeral environment because no system Python ships `numpy` and `pytest` together here. And there is a documented ordering dependency: the rounding gtest *writes* a file the rounding pytest *reads*, so `ctest -R rounding` must run before the pytest tier (as the ctest target ordering above already does). Tier 3 includes the simulator-vs-golden correctness harness (`test_correctness.py`, 11 tests): both targets clear the PCC floor of 0.9875 with non-vacuity defenses (maximum absolute difference strictly positive on every run, fp32-golden vs bf16-simulator structural assertion, and per-target empirical trip-wires).

6.5 Generated-kernel validation mechanics

The generated TT-Metalium kernels are never executed and are validated *structurally only*, by compile-checking them against stub headers:

- **Signature-only stubs, pinned @`df36cd6`.** `kernels/handwritten_baselines/tt_metal_stubs/` mirrors the tt-metal kernel- and host-side API surface the generated kernels use (`dataflow_api.h`, `compute_kernel_api.h` and its `matmul.h`/`pack.h`/`reg_api.h` sub-includes, `host_api.hpp`,

Table 10: Passes of the TensixFlow transform pipeline, one line each, as declared in `TensixflowPasses.td`.

Pass flag	Purpose
`--tensixflow-infer-tile-layout`	Attach a `#tensixflow.tile_layout` encoding to each tensor result (dims rounded to 32×32 tiles, default 8×8 grid).
`--tensixflow-canonicalize`	Canonicalize tensixflow ops: identity-add removal, trivial constant folding, structural dedup.
`--tensixflow-fuse-rmsnorm-matmul`	Rewrite an adjacent rmsnorm→matmul pair into one fused op, cutting intermediate DRAM traffic; the same pass grows the MLP chain into `fused_mlp`.
`--tensixflow-plan-noc-sharding`	Assign tensor shards to cores of the 8×8 grid under the L1 budget and plan NoC movement (consumes `L1Model.h` constants).
`--tensixflow-allocate-circular-buffers`	Allocate L1 circular buffers for the tile streams between reader/compute/writer kernels.
`--tensixflow-lower-to-ttkernel`	Lower fused/sharded ops into the schedule IR (tile-loop nests + CB + NoC + compute ops).
`--tensixflow-emit-metalium`	Emit TT-Metalium-style host/reader/compute/writer `.cpp`; with `output-dir=` write a complete standalone project (+ CMakeLists.txt + README.md).
`--tensixflow-emit-ttir` (compat)	Emit TTIR-shaped MLIR text to stdout — `ttir.*` only (TTNN/D2M stay prose mappings); unmapped ops emit a documented UNSUPPORTED marker, transitively.

Table 11: Validation tiers with current counts (verified live 2026-08-03). C/C++ tiers run under `nix develop --command bash -c '...'` so the pinned LLVM/MLIR 20.1.8 environment is on the PATH.

Tier	Command	Current result
1. lit/FileCheck IR suite	`cmake --build build --target check-tensixflow`	Total 25: **24 passed** + **1 unsupported** (`emit_ttir_ttmlir_parse.mlir`, gated on a `ttmlir-opt` this box does not build)
2. ctest C++ units (gtest)	`cd build && ctest --output-on-failure`	**11/11 passed**: plan_schema, scheduler_model, grid_planner, weight_planner, cb_allocator, noc_estimator, plan_report, rounding, tile_simulator, perf_model, hw-acceptance
3. pytest (Python)	`uv run --with numpy --with pytest pytest python/tensixflow/tests/ -q`	**89 passed** (golden, harness, DSL, lowering, rounding, correctness)
4. generated-kernel compile check	`cmake --build build --target check-generated-kernels-compile`	baseline compiles, legacy `EnqueueProgram` fixture rejected, 4 generated kernels compile (also runs inside ctest's `hw-acceptance`)

`kernel_types.hpp`). Every body is empty ({} / `return {}`) — a compile contract only, no behavior; the headers say so verbatim.

- **The check itself** is orchestrated by `kernels/handwritten_baselines/check_kernels_compile.sh` as the CMake target `check-generated-kernels-compile`: compile each translation unit with `g++ -std=c++20 -fsyntax-only -Wall -I <stubs>`. Syntax-only means no link step and no device is possible — but the compiler still resolves every API name and signature the kernels use.
- **Non-vacuous by construction.** The target asserts three things, all required: (1) *positive* — the current-API baseline kernel compiles, proving the stubs still carry the current symbols; (2) *negative* — a fixture referencing the removed legacy symbol `EnqueueProgram` must *fail* with a diagnostic naming that symbol, proving the stubs have not drifted back onto the legacy API; (3) every generated kernel file compiles. If the check ever passed vacuously, assertion (2) would fail instead.

7 Deployment

Deployment, for TensixFlow, means reproducibility of the toolchain plus two CI paths: an always-on CPU path and a double-gated hardware path. There is no cloud provisioning anywhere in the loop.

7.1 Reproducibility: the pinned nix toolchain

The development environment is a nix flake (`flake.nix` + committed `flake.lock`):

- **Pinned input.** `flake.lock` locks `NixOS/nixpkgs` to rev `bd3c09ca2a2858daa15f26eb557faa4709635a41` (`nixos-unstable`), so every shell resolves the same package set on every machine.
- **Pinned LLVM.** `flake.nix` selects the explicit `llvmPackages_20` set — LLVM/MLIR 20.1.8 — and its comment forbids the floating default `llvmPackages`. The shell exports `LLVM_DIR`/`MLIR_DIR` from the pinned store paths and puts `mlir-tblgen` on the PATH; the package list also fixes cmake, ninja, lit, and GoogleTest (with the `dev` output named explicitly so `find_package(GTest)` resolves).
- **devenv layering.** `devenv.yaml` re-pins its `nixpkgs` input to the *same* rev (`bd3c09ca2a2858daa15f26eb557faa4709635a41`) with an explicit warning: at a floating `nixos-unstable` URL, `devenv shell` re-resolves the lock to a head whose mlir is uncached and source-broken. The rev must stay pinned here, not only in `devenv.lock`.
- **The lit pin.** Configure passes `-DLLVM_EXTERNAL_LIT=$(dirname $(which lit))/.lit-wrapped`: under nix, `bin/lit` is a bash wrapper, and LLVM's `add_lit_target` spawns lit as `python3 <lit>`, which breaks on the wrapper. Pointing CMake at the real Python script `.lit-wrapped` keeps the lit target working.

The exact configure command, as documented in the README:

```
cmake -G Ninja -B build \
  -DMLIR_DIR=$MLIR_DIR -DLLVM_DIR=$LLVM_DIR \
  -DLLVM_EXTERNAL_LIT="$(dirname "$(which lit)")/.lit-wrapped"
```

One sentence on this document itself: the LaTeX report builds in an ephemeral `nix shell nixpkgs#texlive.combined.scheme-full` (TeX Live 2025) that resolves through the same pinned registry input, without touching the dev flake — see `report/BUILDING.md`.

7.2 CI topology

Two workflows, two disjoint runner classes (Figure 9).

Notes on the CPU path (all from `.github/workflows/ci.yml`): the ctest step passes `--no-tests=ignore` and the pytest step tolerates exit code 5 ("no tests ran") so both stayed green in early waves; today both carry real suites. The generated-kernel compile check is not a separate CI step — it is enforced inside ctest via `hw-acceptance`, whose no-device branch runs `scripts/hw_bringup_dryrun.sh&&cmake--buildbuild--targetcheck-generated-kernels-compile` (Section 5.4).

7.3 Cloud policy: opt-in, documented, never invoked

The MVP is local-first by policy:

- `docs/remote-box.md` states it up front: "You do not need this." A remote Linux build box is *purely opt-in*, for keeping a laptop free; the document's provider notes (e.g. PrimeIntellect CPU sandboxes as the cheapest option) are indicative price ordering, to be verified in-console.

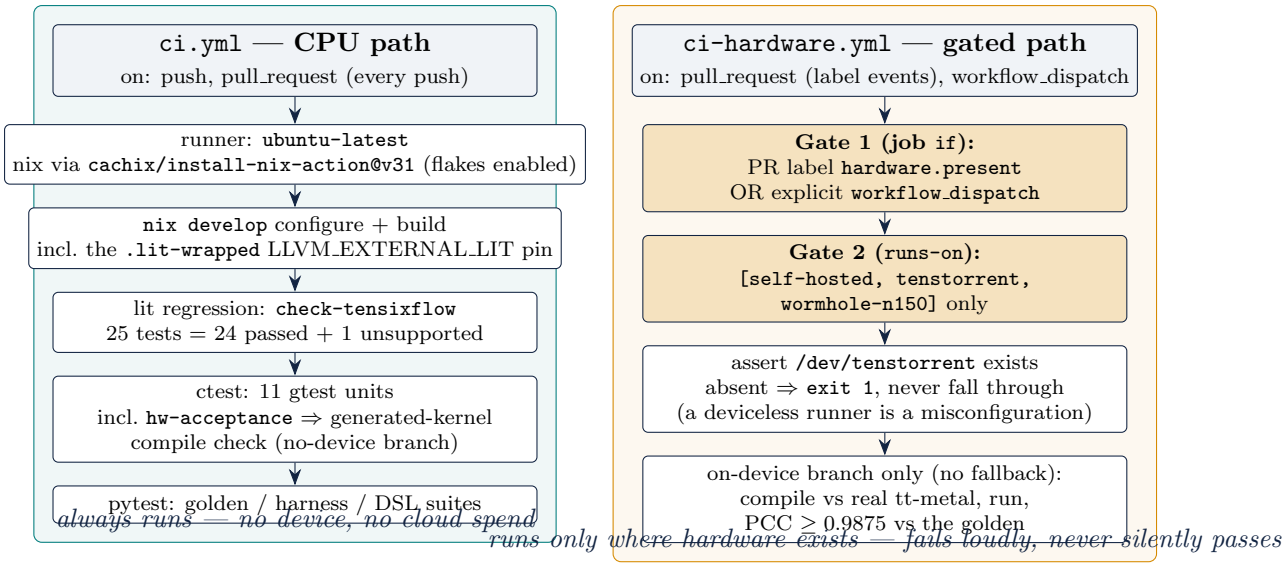


Figure 9: CI topology: the CPU path (`ci.yml`) runs the full build/lit/ctest/pytest chain on every push, with the generated-kernel compile check riding inside ctest's `hw-acceptance` test; the hardware path (`ci-hardware.yml`) is double-gated (Section 5.4) and only ever runs on a tagged self-hosted runner carrying the device.

- `scripts/remote-box.sh` is an idempotent setup script run *manually* on a machine the user already owns access to: install nix if absent, clone the repo, enter the dev shell, run the standard configure + build. It provisions nothing and is never invoked by CI.
- Nothing in `ci.yml`, `ci-hardware.yml`, or the test suites provisions or contacts a cloud provider, and nothing incurs cloud spend automatically. Likewise `scripts/build-tt-mlir.sh` (the optional tt-mlir study helper) is off the critical path: no CMake target depends on it.

7.4 Upstream contribution: scouted, implemented locally, not submitted

Status, exactly: TensixFlow *scouted* a concrete tt-mlir contribution and *implemented it locally*; the submission process is fully documented; and **nothing was submitted, forked, pushed, or merged** — per the project owner's explicit binding instruction. This is policy in action, not an omission:

- **Scouted** (`docs/upstream_contribution.md`, todo 58): one new lit test for tt-mlir's `ttir.softmax` verifier, covering both error branches — one of which (shape mismatch) had *zero* test coverage anywhere in tt-mlir at the scouted revision (`main` @ `da6f1ce0`). The gap is cited from the actual repo: verifier source, op definition, tree listings, and a search over the diagnostic strings.
- **Implemented locally** (`upstream/tt-mlir/`, todo 59): `softmax_tests_negative.mlir` — 22 lines, three negative cases, CHECK strings quoted verbatim from `SoftmaxOp::verify()` — plus a README recording provenance (scratch clone, *not* a fork; branch `softmax-negative-tests` based on `da6f1ce0`; upstream lint hooks run; scoped diff of exactly one new file) and an honest "what is NOT verified" section (never executed against a real `ttmlir-opt`, whose toolchain is not built on this box).
- **Submission documented, never executed**: `docs/upstream_contribution.md` records the complete process (fork + branch, `pre-commit`, targeted `llvm-lit`, PR template, CI expectations) under a heading that says "DOCUMENTED ONLY; never executed in this plan," and states that the plan "explicitly stops before step 1." The same constraint is repeated in `upstream/tt-mlir/README.md`.

8 Appendix

8.1 Verification commands (F3 final wave)

Table 12 reproduces the exact command sequence of the F3 final verification wave (`.omo/evidence/task-F3-tensixflow.log`, 2026-08-03, CPU-only box). Every step exited 0. The wave additionally recorded: environment probes confirming `/dev/tenstorrent` absent and no GPU present; emitted project contents for both examples (host/reader/compute/writer.cpp + CMakeLists.txt + README.md); regenerated report artifacts byte-identical to the committed ones; captured PCC values for both targets on multiple seeds (all ≥ 0.99998, floor 0.9875); and the offline HTML render check under a network-blocked browser (zero requests, all six section anchors rendered).

Table 12: Exact F3 final-wave command sequence with exit codes (all 0).

#	Exact command	Result (rc)
1a	`nix develop --command bash -c 'cmake -G Ninja -B build -DMLIR_DIR=$MLIR_DIR -DLLVM_DIR=$LLVM_DIR -DLLVM_EXTERNAL_LIT=$(dirname $(which lit))/.lit-wrapped'`	configure rc=0 (MLIR 20.1.8 pinned)
1b	`nix develop --command bash -c 'cmake --build build'`	build rc=0
2	`nix develop --command bash -c 'cmake --build build --target check-tensixflow'`	rc=0; 25 = 24 passed + 1 unsupported
3	`nix develop --command bash -c 'cd build && ctest --output-on-failure'`	rc=0; 11/11 passed
4	`uv run --with numpy --with pytest pytest python/tensixflow/tests/ -q`	rc=0; 89 passed
5	`build/bin/tensixflow-opt --tensixflow-infer-tile-layout --tensixflow-fuse-rmsnorm-matmul --tensixflow-plan-noc-sharding --tensixflow-allocate-circular-buffers --tensixflow-lower-to-ttkernel --tensixflow-emit-metalium=output-dir=<dir> examples/rmsnorm_matmul.mlir` `# same command for examples/mlp_block.mlir`	rc=0 both examples; 6 files emitted each
6	`build/bin/tensixflow-opt --tensixflow-plan-noc-sharding --tensixflow-allocate-circular-buffers examples/rmsnorm_matmul.mlir | build/bin/tensixflow-translate --tensixflow-to-json > reports/rmsnorm_matmul_plan.json` `build/bin/tensixflow-report --in reports/rmsnorm_matmul_plan.json --out reports/rmsnorm_matmul_report.html`	rc=0; HTML regenerated byte-identical
7	`uv run --with numpy --with pytest pytest python/tensixflow/tests/test_correctness.py -v`	rc=0; 11/11; all PCC ≥ 0.9875, tripwires hold

Table 12 (continued)

#	Exact command	Result (rc)
8	`nix develop --command bash -c 'bash scripts/hw_acceptance.sh'`	rc=0; no-device branch: dry-run + 4 generated kernels compile, legacy rejected
9	`uv run --with playwright python f3_offline_html_check.py`	rc=0; zero network requests, 6/6 anchors rendered

Steps 1–4 of Table 12 were re-executed for this report with identical outcomes (lit 25 = 24 + 1 unsupported; ctest 11/11; pytest 89 — Section 6.4).

8.2 Glossary

Table 13: Glossary of TensixFlow terms.

Term	Definition
Circular buffer (CB)	A per-core L1 FIFO of tiles staging data between the reader, compute, and writer tile programs. Sized depth × `kTileBytes` (a depth-160 buffer occupies 160 × 2048 = 327,680 B); indexed per the tt-metal `tt::CBIndex` convention `c_0`…`c_31` (default pipeline: in0=`c_0`, in1=`c_1`, out=`c_16`); synchronized via `cb_reserve_back`/`cb_push_back` (producer) and `cb_wait_front`/`cb_pop_front` (consumer).
NoC	Network-on-chip: the Wormhole grid's asynchronous interconnect. TensixFlow plans per-core shard placement and tile movement over it (`noc_async_read`/`noc_async_write`, plus multicast to alleviate DRAM bandwidth contention) and estimates the resulting DRAM traffic in the plan.
PCC	Pearson correlation coefficient. The primary numerical metric comparing a result tensor against the fp32-exact NumPy golden; the acceptance floor is 0.9875 in both the simulator harness and the (gated) on-device branch.
Schedule IR	The tile-level `tensixflow.sched.*` dialect: a `sched.kernel` holding reader/compute/writer tile programs built from `tile_loop` nests plus CB, NoC, and compute ops. It is the shared contract the emitter prints and the functional simulator executes (Section 6.1).
Shard	The slice of a tensor assigned to one core's L1 by `tensixflow-plan-noc-sharding`, placed under the modeled 960 KiB CB/shard budget and routed over the NoC.
Tile	The 32 × 32-element unit of data: bf16 tiles are 32 × 32 × 2 = 2048 B. Tiles are the unit of CB depth, NoC moves, and compute ops.
Trip-wire	An empirically calibrated PCC threshold set *above* the good case's observed PCC, so a regression trips the wire before it ever approaches the 0.9875 acceptance floor (`test_correctness.py`).

Table 13 (continued)

Term	Definition
Stub headers	Signature-only mirrors of the tt-metal API surface (`kernels/handwritten_baselines/tt_metal_stubs/`): correct names and signatures, every body empty, pinned to tt-metal `@df36cd6`. A compile contract only — no behavior.
Structural-only validation	Validating generated kernels without execution: golden FileCheck patterns against the current tt-metal API plus the `-fsyntax-only` compile check against the stubs. Never run on hardware; no hardware numerics claimed.
Disjunctive acceptance	The hardware acceptance policy of Section 5.4: (on-device run AND PCC $\geq$ 0.9875) OR (validated no-device substitute). Wired as an explicit `if/else`, never `&&...||`, so a present-but-failing device can never be masked by the passing no-device branch.

8.3 Deviation register

Documented deviations from the original plan — notably the no-device hardware posture (todo 55's on-device entry point `SKIPPED`: no Tenstorrent device is owned) and the deliberately unsubmitted upstream contribution (todo 60 `SKIPPED` per explicit user instruction) — are recorded with their rationale in the decisions section (Section 2). Both are policy-sanctioned: each has a validated substitute (the no-device branch, which passes; the local implementation plus submission documentation) rather than a silent gap.